Everclear Swaps & Hub Upgrade

September 2025

Made with ❤ by the following Creed authors: Valentin Quelquejay and Dominik Muhs

Table of Contents

Executive Summary 4

Scope and Objectives 5

Audit Artifacts 6

Findings 7

- [Major] Potential Loss of Funds When Settling on Non-EVM Chains
- [Minor] Solver Destination Preferences Cause Stuck Invoices
- [Minor] Remove Deprecated Gas Config

Disclaimer 13

Executive Summary

This report presents the results of our engagement with Everclear to review their Hub Gateway mailbox upgrade as well as their upcoming swaps feature.

The review was conducted over two weeks, from September 15, 2025 to September 26, 2025 by Valentin Quelquejay and Dominik Muhs. A total of 20 person-days were spent.

Overall, three findings have been identified, one major and two of minor severity. The major finding relates to the invalid handling of solver addresses on non-EVM settlement domains, which results in a loss of funds. Another significant minor finding is a corner case where solver destination preferences are handled incorrectly by the Hub.

We also checked the fixes implemented by the Everclear team and confirmed that all issues were considered and addressed. As the fixes were minor and localized, no additional follow-up engagement was scheduled.

Scope and Objectives

Our review focused on the following commit hashes:

- Hub Gateway Upgrade: `026cd86cb5867126530409ce8db741a815f77001`
- Swaps: `14d2fd9609817856cc381f80919e96327cee9d8e`

Following completion of the review, the Everclear team requested that we extend our scope to include commit `5eb4cf2897583ff32967d3f24c51dd821a06bd72`.

Together with the Everclear team, we identified the following priorities for our review:

- Prioritize the Hub upgrade due to time constraints.
- Ensure that the system is implemented consistently with the intended functionality, and without unintended edge cases.
- Identify known vulnerabilities particular to smart contract systems, as outlined in our Smart Contract Security Field Guide, and the ones outlined in the EEA EthTrust Security Levels Specification.

Audit Artifacts

Hub Gateway Upgrade

The Hub Gateway Mailbox Upgrade proposes a set of minor changes to the Everclear Hub's Gateway infrastructure to add support for alternative messaging providers such as Polymer, LayerZero, or Hyperlane per supported domain.

These changes only affect the gateway base contract (now `GatewayV2`) and its Hub instance (now `HubGatewayV2`). All original function interfaces are kept intact so that in-flight messages will not immediately fail. We would, however, like to highlight a potential risk for message loss regarding the Gateway's `handle` function, which is called by the mailbox for incoming messages. The authorization logic of `handle` has been updated to authenticate different senders based on their origin domain dynamically. Based on the domain, messages in transit could be discarded by the `HubGatewayV2` contract and must be replayed by the Everclear team to maintain a consistent record on the Hub. We recommend pausing domains where the mailbox is different post-upgrade, flushing their message queues, and then performing the upgrade to avoid message loss.

We furthermore recommend monitoring messages and queue invariants during the upgrade to detect potential issues early, before they can further impact the system's tracking of funds.

Swaps Upgrade

This upgrade introduces a swaps feature: users can submit intents with different input/output assets, solvers can fill them and be settled in the input asset on any approved domain. The feature is restricted to a whitelisted set of tokens. Only tokens with strict, plain ERC-20 semantics should be admitted. Tokens with fees on transfer, rebasing/deflationary mechanics, or non-standard pre/post-transfer hooks can distort amounts, break settlement expectations, or cause shortfalls for solvers and users. We recommend formal due diligence before listing (e.g., checklist, code review, and transfer simulations) to preserve predictable settlement outcomes.

Diverging Releases

The code base under review is based on the v1.1 release (Chimera). It should be noted that the v1.2 (Diablo) release introduces fundamental changes to how intents are processed, filled, and how solvers and users interact with the system.

Additional changes introduced in Chimera will eventually need to be merged into the Diablo release. The introduction of interchangeable transport providers and swaps affects code areas that the Diablo release has significantly changed. Thus, we recommend a follow-up review and utmost caution to avoid any unintended side effects during the refactor.

Findings

Major Potential Loss of Funds When Settling on Non-EVM Chains

Partially Fixed

> This issue was identified collaboratively by the Everclear development team and our audit team during review discussions. It was partially addressed in commit `58aae510147e430aecfdf16273ff6ab27a3e968b`, which ensures that intents are validated off-chain. Note that it is important to also verify off-chain that solvers' preferences stored in the hub do not conflict or overwrite intent destinations with invalid values.

When filling intents, solvers can choose the settlement domain—they do not need to be settled on the chain where they filled the intent. This design enables Everclear's global clearing layer. When fill messages are processed on the Hub, `_processFillMessages` calls `_createSettlementOrInvoice` to either settle the solver or create an invoice if immediate liquidity is unavailable.

In `_createSettlementOrInvoice`, the `_recipient` parameter is set to `_fillMessage.solver`, which is the solver's address on the chain where `fillIntent` was called. On the Spoke, `fillIntent` sets `_solver` to `msg.sender`:

swaps/packages/contracts/src/contracts/intent/EverclearSpokeV5.sol

```
/// @inheritdoc IEverclearSpokeV5
function fillIntent(
  Intent calldata _intent,
  uint256 _amountOut,
  uint32[] memory _destinations
) external whenNotPaused returns (FillMessage memory _fillMessage) {
  _fillMessage = _fillIntent(_intent, msg.sender, _amountOut, _destinations, fal…
}
```

`_processFillMessages` sets `_recipient` to `_fillMessage.solver`, i.e., the address used by the solver that filled the message:

swaps/packages/contracts/src/contracts/hub/modules/HubMessageReceiverV2.sol

```
if (_previousStatus == IntentStatus.DEPOSIT_PROCESSED) {
  Intent memory _intent = _contexts[_intentId].intent;
  bytes32 _tickerHash = _adoptedForAssets[AssetUtils.getAssetHash(_intent.inputA…
  // settle solver
  _createSettlementOrInvoice({_intentId: _intentId, _tickerHash: _tickerHash, _r…
} else {
  _intentContext.status =
```

```
    _previousStatus == IntentStatus.ADDED ? IntentStatus.ADDED_AND_FILLED : Inte…
}
```

`_createSettlementOrInvoice` then creates a settlement or, if needed, an invoice with `_recipient` (resp. `_owner`) set to `recipient`:

swaps/packages/contracts/src/contracts/hub/modules/SettlerLogicV2.sol

```
if (_selectedDestination != 0) {
  _createSettlement({
    _intentId: _intentId,
    _tickerHash: _tickerHash,
    _amount: _amountAndRewards,
    _destination: _selectedDestination,
    _recipient: _recipient
  });
} else {
  // Send to invoice queue debt
  _createInvoice({_tickerHash: _tickerHash, _intentId: _intentId, _amount: _amou…
```

All addresses are handled as `bytes32` on the Hub to support both EVM and non-EVM chains. If a solver fills on an EVM chain and selects a non-EVM settlement domain (or vice versa), the solver's origin-chain address is not valid on the destination chain. In this case, the settlement targets an unusable address, and the funds will be lost on the settlement domain.

Recommendation

Allow solvers to specify a per-destination repayment address, especially for non-EVM chains, rather than implicitly reusing the origin-chain address. Ensure the chosen repayment address is included in the relevant signatures for relayer processing.

Minor Solver Destination Preferences Cause Stuck Invoices

Fixed

> This issue was addressed by the team via commit `794184bededb98592e0f3fcd674b508ccf54cd95`.

On the Spoke, a solver must provide a `_destinations` array to `fillIntent`, indicating the domains where they wish to be settled. Because swaps are always repaid in the **input** asset, when the fill message reaches the Hub, it checks whether the solver-provided destinations support the input asset; if none do, the repayment domain defaults to the swap's origin.

swaps/packages/contracts/src/contracts/hub/modules/HubMessageReceiverV2.sol

```
bool supportedDestinations =
  _checkSupportedDestinations(_fillMessage.intentInputAsset, _fillMessage.intent…
if (supportedDestinations) {
  _intentContext.solverDestinations = _fillMessage.destinations;
} else {
  _intentContext.solverDestinations = new uint32[](1);
  _intentContext.solverDestinations[0] = _fillMessage.intentOrigin;
}
```

However, during settlement creation, the Hub resolves destinations via `_getDestinations`, which prioritizes user-configured domains (`_usersSupportedDomains`) over the solver-provided (and already validated) destinations. This effectively bypasses the earlier asset-support check. The function `setUserSupportedDomains` does not verify that a domain supports all the `assetHash`.

swaps/packages/contracts/src/contracts/hub/modules/managers/UsersManagerV2.sol

```
function setUserSupportedDomains(
  uint32[] calldata __supportedDomains
) external {
  if (__supportedDomains.length < minSolverSupportedDomains) {
    revert UsersManager_SetUser_MinimumSupportedDomainsNotMet(minSolverSupported…
  }
  bytes32 _user = msg.sender.toBytes32();

  Uint32Set.Set storage _userSupportedDomains = _usersSupportedDomains[_user];

  if (_userSupportedDomains.length() > 0) {
    _userSupportedDomains.flush();
  }

  for (uint256 _i; _i < __supportedDomains.length; _i++) {
    if (!_supportedDomains.contains(__supportedDomains[_i])) {
      revert UsersManager_SetUser_DomainNotSupported(__supportedDomains[_i]);
    }
    if (!_userSupportedDomains.add(__supportedDomains[_i])) {
      revert UsersManager_SetUser_DuplicatedSupportedDomain(__supportedDomains[_…
```

```
    }
  }

  emit SolverConfigUpdated(_user, __supportedDomains);
}
```

Consequently, if a solver has `_usersSupportedDomains` that lack support for the swap's input asset, `_createSettlementOrInvoice` will enqueue an invoice that can never be settled because `_findDestinationWithStrategies` will not return a valid destination. The invoice remains stuck in the pending queue until the solver reconfigures their preferred domains.

swaps/packages/contracts/src/contracts/hub/modules/SettlerLogicV2.sol

```
function _findDestinationWithStrategies(
  bytes32 _tickerHash,
  uint256 _amountAndRewards,
  uint32[] memory _destinations
) internal view returns (uint32 _selectedDestination) {
  if (_tokenConfigs[_tickerHash].prioritizedStrategy == IEverclearV2.Strategy.XE…
    _selectedDestination = _findDestinationXerc20Strategy(_tickerHash, _destinat…
    if (_selectedDestination == 0) {
      _selectedDestination = _findDestinationDefaultStrategy(_tickerHash, _amoun…
    } // else if will be added for other strategies in order of priority
  } else {
    _selectedDestination = _findDestinationDefaultStrategy(_tickerHash, _amountA…
    if (_selectedDestination == 0) {
      _selectedDestination = _findDestinationXerc20Strategy(_tickerHash, _destin…
    }
  }
}
```

Recommendation

There are several ways to mitigate this issue:

1. Bypass `_usersSupportedDomains` for swaps. Since solvers must provide destinations, use the solver-validated list (or the origin fallback) for swaps, ignoring user-configured domains.
2. When reading `_usersSupportedDomains` in `_getDestinations`, validate against domains that support the input asset's `assetHash`; if none remain, fall back to the origin domain (for swaps).

Minor

Remove Deprecated Gas Config

Fixed

> This issue was addressed by the team via commit `9bf4391d623f794b53c99affccee263475669fed`.

The V2 upgrade now passes a gas limit computed off-chain into the `processSettlementQueue()` function in the `SettlerV2` contract, instead of computing it within the contract. Thus, the internal gas configuration is no longer used. However, due to the upgrade pattern, it remains exposed and can be updated.

`updateGasConfig` in `EverclearHubV2`:

swaps/packages/contracts/src/contracts/hub/EverclearHubV2.sol

```
  function updateGasConfig(
    GasConfig calldata
  ) external {
    _delegate(_MANAGER_MODULE);
  }
```

`updateGasConfig` in `ProtocolManagerV2`:

swaps/packages/contracts/src/contracts/hub/modules/managers/ProtocolManagerV2.sol

```
function updateGasConfig(
  GasConfig calldata _newGasConfig
) external hasRole(Role.ADMIN) {
  if (_newGasConfig.bufferDBPS > Common.DBPS_DENOMINATOR) {
    revert HubStorage_InvalidDbpsValue();
  }
  GasConfig memory _oldGasConfig = gasConfig;
  gasConfig = _newGasConfig;
  emit GasConfigUpdated(_oldGasConfig, _newGasConfig);
}
```

`GasConfig` struct/storage in `IHubStorageV2`:

swaps/packages/contracts/src/interfaces/hub/IHubStorageV2.sol

```
/**
 * @notice Struct for settlement message gas configuration
 * @param settlementBaseGasUnits The amount of base gas units for a settlement m…
 * @param averageGasUnitsPerSettlement The average amount of gas units per settl…
 * @param bufferDBPS The gas buffer for relay on destination (in DBPS)
 */
struct GasConfig {
  uint256 settlementBaseGasUnits;
  uint256 averageGasUnitsPerSettlement;
```

```
    uint256 bufferDBPS;
}
```

Recommendation

As the `gasConfig` struct in the hub storage cannot be removed to avoid breaking the contract's storage layout, we recommend marking it as deprecated along with `updateGasConfig()`, and making `updateGasConfig()` revert to prevent accidental use.

File Hashes

Hub Upgrade

- gateway-mailbox/packages/contracts/src/contracts/hub/HubGatewayV2.sol
    - bbfe2a66af93e554c8d8165a7041e1b11131c57b7e6b7fcfa92bd7057efef3d6
- gateway-mailbox/packages/contracts/src/contracts/common/GatewayV2.sol
    - 67a720f369e624645fc3b763a821bcce411d36e6859c5e1154b6d38f6153f292

Swaps

- swaps/packages/contracts/src/contracts/hub/lib/Uint32Set.sol
    - 8174ee18301d7ad195a5f32e7d62871a38e94be7205fb95109278f6c81b9a675
- swaps/packages/contracts/src/contracts/hub/lib/InvoiceListLibV2.sol
    - 9ba519a3a7f8c0d60bde98e8b6dfa4450d5ac011f10c4da179821d440e4ed03b
- swaps/packages/contracts/src/contracts/hub/lib/InvoiceListLib.sol
    - fa5effd48f933e5609b9a5769c1c717d1a02b76687a3d3f1bc0e9adc66332e3e
- swaps/packages/contracts/src/contracts/hub/lib/HubQueueLib.sol
    - 1de362b5c84c5780ce9721af11ac5461ea25b955bfcb5ec2b46b8c0deaf59b69
- swaps/packages/contracts/src/contracts/hub/lib/HubQueueLibV2.sol
    - 72e47d4d545fe6bf15a555a496c6089311833a97b6526323f95d44e49cbbefcd
- swaps/packages/contracts/src/contracts/hub/HubGateway.sol
    - 840a86a503adcdd086f7652ccefeafee0302f060c59a789270f5d9544e8b3ffa
- swaps/packages/contracts/src/contracts/hub/HubStorageV2.sol
    - 3c377978f262544b1b018bc4ec866c5364e67304fd76db2aee4ecec97b083a45
- swaps/packages/contracts/src/contracts/hub/HubStorage.sol
    - bbb44611da7ff6bf704010366cb1564be8b20cec2aa27c6a060a7fccb834b425
- swaps/packages/contracts/src/contracts/hub/modules/HandlerV2.sol
    - 55731c091b700bc7b9762ec34cf4e328521491d46f5473e2581ea4983cdf856a
- swaps/packages/contracts/src/contracts/hub/modules/SettlerLogic.sol
    - de59eeff2b29a7aa131cc87d384019352032baf9e9ec95ea2cda198c12a575ca
- swaps/packages/contracts/src/contracts/hub/modules/Handler.sol
    - b30a06beadd4dc1c7530930d0f911a102af512edd4212d4019b7e0c9d781b4a2
- swaps/packages/contracts/src/contracts/hub/modules/Manager.sol
    - 35918baa95dbaf2356faf5784d2135f89bf56c047f0e39b90bac4a270d845b9b

- swaps/packages/contracts/src/contracts/hub/modules/SettlerV2.sol
  - 583f8340599b192a489f9cfb7cab44b7ad3eb62cfb16a3e3fc7e6403929892b9
- swaps/packages/contracts/src/contracts/hub/modules/HubMessageReceiverV2.sol
  - 5f1329aeaae72ea940aab99a0fd2e55a202d9531141091fa341c8b9767651f5b
- swaps/packages/contracts/src/contracts/hub/modules/ManagerV2.sol
  - ac2c6ce3c7139bd647df5a9f3339e8fa2be3711d2713bb6cb9b4bf74b80734b8
- swaps/packages/contracts/src/contracts/hub/modules/SettlerLogicV2.sol
  - e46b765cd74fc345201426c9ff887e7c71b0a60231592fb45bc16c8bb79a41d0
- swaps/packages/contracts/src/contracts/hub/modules/HubMessageReceiver.sol
  - 99cb474d1b8e3506e14ddf0b5f773bd0f12bf097131f029876741db6eb2e2205
- swaps/packages/contracts/src/contracts/hub/modules/managers/ProtocolManager.sol
  - 809b0a8f9a1d23fe99526e8207775a2f6f6cfc753ed6b6dfb00ade658012d3b2
- swaps/packages/contracts/src/contracts/hub/modules/managers/AssetManager.sol
  - 2d23a14819dd140e8180ae2c1aeef1e5afb04cb239b0349a6563803851701b57
- swaps/packages/contracts/src/contracts/hub/modules/managers/UsersManagerV2.sol
  - 2b7cdc577fc799e53ac382c528c10d0a23ffd0e614cd9aba368e207770780fe4
- swaps/packages/contracts/src/contracts/hub/modules/managers/ProtocolManagerV2.sol
  - 81386439fc6680508bdf10986961b8f0395f5192e3ee7bbcd9aac4f14f67ac0a
- swaps/packages/contracts/src/contracts/hub/modules/managers/UsersManager.sol
  - 1442b7ea09328ce3ae4a40753b4325fd79746c1c34f94db80874d1f5411c7419
- swaps/packages/contracts/src/contracts/hub/modules/managers/AssetManagerV2.sol
  - 61ace02993c3349b5f958908144157c219c72d9588bb0961c12357bc5ae8a4da
- swaps/packages/contracts/src/contracts/hub/modules/Settler.sol
  - 2fff85ebb0680c01976c2132ee0fb6283a26bde0cae86bceca95d4071ad40953
- swaps/packages/contracts/src/contracts/hub/EverclearHubV2.sol
  - f532bffd76d5fb57a3192fb4f77a42666526c9695052b272dcab343628e980b2
- swaps/packages/contracts/src/contracts/hub/EverclearHub.sol
  - 304a7fde6a1e24dad9deefd0789b15058bc2e2c0f28bf13ed9cfec116e62ff81
- swaps/packages/contracts/src/contracts/common/QueueLib.sol
  - a0216277e304e4d531bfba3dbac39f6542a931ddf7730edb1db24863f79d825e
- swaps/packages/contracts/src/contracts/common/Constants.sol
  - 35c4a3218a9bfa9096aaec0463c66af6577f86b59b41df3c04439b90dac7ef09
- swaps/packages/contracts/src/contracts/common/AssetUtils.sol
  - 92d125eb34a6f742840d51ef5ad1e5aa9890f75c1c20f143a7a4148b6e424a8b

- swaps/packages/contracts/src/contracts/common/GasTank.sol
  - 91aba30718ec1ce3a353e503f94aeb25258b31d811d27ba94fdc11ab7facc2b9
- swaps/packages/contracts/src/contracts/common/MessageLib.sol
  - 2ee036d4fb34d86923815ac765125f604dbd0ffe82ee2744c63053e2e2bf350a
- swaps/packages/contracts/src/contracts/common/QueueLibV2.sol
  - 32c529f78ddb4aabd1b844d6feb6fa7d97b393bf77157669fb8e80672870ebe6
- swaps/packages/contracts/src/contracts/common/Gateway.sol
  - 21d64c829f40c787e64a0f3b82a820cef7f8ac84eb05c632958dd51ce6194ac9
- swaps/packages/contracts/src/contracts/common/TypeCasts.sol
  - 6b1989e2e8c861f4c67c7b52340be92c88e8a48bf18aaf5c896fa74298d8f65e
- swaps/packages/contracts/src/contracts/common/MessageLibV2.sol
  - 3d42d8ed5cbdacc2a9db38a1f77cd5a19f303152219d3885c1d9973ef856ab7c
- swaps/packages/contracts/src/contracts/intent/EverclearNanoSpoke.sol
  - b82d455389281fa6c8ef28a9362ee426aba0e7060a86a99f035eb305a8eeb2a8
- swaps/packages/contracts/src/contracts/intent/SpokeStorage.sol
  - 1c720fcc44d5b8a7f613beac162b5712d6930be77d88c14cc20eff7d725d5fbb
- swaps/packages/contracts/src/contracts/intent/SpokeGateway.sol
  - ffb83ea0686f83c4074e05eed2477571df4c610a9fc423f61be0f7cb97efdd2f
- swaps/packages/contracts/src/contracts/intent/lib/Constants.sol
  - e2e4f3da93532ca94faf9684c601e5127de27598504268f7c16f8829fe3dc119
- swaps/packages/contracts/src/contracts/intent/FeeAdapter.sol
  - 12319f2f22262986df40e7afb762a39d0a837474fc407c03dc438e76b6b32a02
- swaps/packages/contracts/src/contracts/intent/EverclearSpokeV4.sol
  - bb0b7fba28db5f0096564a5f7fcc292004eac6700681ac3f65dcf4a8de9ee8f0
- swaps/packages/contracts/src/contracts/intent/CallExecutor.sol
  - 2e8ca8e32d93a1cc7eedd9ec3b09d6fd6c6b608d427a24c35ed056a12d688aba
- swaps/packages/contracts/src/contracts/intent/EverclearSpokeV3.sol
  - 47d452f5f3a2eba476db1e9508f2283514011dd7b41a8aed414890854bd9a71f
- swaps/packages/contracts/src/contracts/intent/SpokeStorageV4.sol
  - ccde8aefa4a93ed1404dbdf91d7ecdb26f118afaf4c065c7c9e1a2c651323ddd
- swaps/packages/contracts/src/contracts/intent/EverclearSpokeV5.sol
  - 7bc13c86902095ce926e1e6550e8b30680959e18c260b94a923efc9aecb1e1aa
- swaps/packages/contracts/src/contracts/intent/modules/XERC20Module.sol
  - f5502eb5adbcfd0d7dbbdc210271863d994291b52d25f8d30bfd4c42d154ce71
- swaps/packages/contracts/src/contracts/intent/modules/SpokeMessageReceiver.sol
  - f5dfbdf982069118e85f5ef33a1ed4b091ea0a3aea55d8f3c93e1289513ae5ce
- swaps/packages/contracts/src/contracts/intent/modules/SpokeMessageReceiverV2.sol
  - afc918630b9674e8828646b39f88dc23562e9130fdee0f1e37cdc684e076c52d

- swaps/packages/contracts/src/contracts/intent/SpokeStorageV5.sol
  - 9cea6c3d83851629b681fb9d07ea19fc91f2873ab1af3aca80b38a5467f314cb
- swaps/packages/contracts/src/contracts/intent/EverclearSpoke.sol
  - 179182d4af34684cfe8003c0b793a885840507a9799c1eacfd5f91125dc9dbd3
- swaps/packages/contracts/src/contracts/intent/FeeAdapterV2.sol
  - cb55529939fd488d2bdf69661fa4315ab975c49a4f229ba8b17d06e1d55dfe14

Disclaimer

Creed ("CD") typically receives compensation from one or more clients (the "Clients") for performing the analysis contained in these reports (the "Reports"). The Reports may be distributed through other means, including via CD publications and other distributions.

The Reports are not an endorsement or indictment of any particular project or team, and the Reports do not guarantee the security of any particular project. This Report does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. No Report provides any warranty or representation to any Third-Party in any respect, including regarding the bugfree nature of code, the business model or proprietors of any such business model, and the legal compliance of any such business. No third party should rely on the Reports in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset. Specifically, for the avoidance of doubt, this Report does not constitute investment advice, is not intended to be relied upon as investment advice, is not an endorsement of this project or team, and it is not a guarantee as to the absolute security of the project. CD owes no duty to any Third-Party by virtue of publishing these Reports.

PURPOSE OF REPORTS The Reports and the analysis described therein are created solely for Clients and published with their consent. The scope of our review is limited to a review of code and only the code we note as being within the scope of our review within this report. Any Solidity code itself presents unique and unquantifiable risks as the Solidity language itself remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond specified code that could present security risks. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. In some instances, we may perform penetration testing or infrastructure assessments depending on the scope of the particular engagement.

CD makes the Reports available to parties other than the Clients (i.e., "third parties") – on its website. CD hopes that by making these analyses publicly available, it can help the blockchain ecosystem develop technical best practices in this rapidly evolving area of innovation.

LINKS TO OTHER WEB SITES FROM THIS WEB SITE You may, through hypertext or other computer links, gain access to web sites operated by persons other than CD. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that CD are not responsible for the content or operation of such Web sites, and that CD shall have no liability to you or any other person or entity for the use of third party Web sites. Except as described below, a hyperlink from this web Site to another web site does not imply or mean that CD endorses the content on that Web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the Reports. CD assumes no responsibility for the use of third party software on the Web Site and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any outcome generated by such software.

TIMELINESS OF CONTENT The content contained in the Reports is current as of the date appearing on the Report and is subject to change without notice. Unless indicated otherwise, by CD.